Team Notebook

October 13, 2026

Contents

1 DataStructure **2**
- 1.1 dsurollback . . . 2
- 1.2 flow . . . 2
- 1.3 fw2d . . . 2
- 1.4 hld . . . 3
- 1.5 lichao . . . 3
- 1.6 pbds . . . 3

2 math **4**
- 2.1 $_{i}nt128$ . . . 4
- 2.2 crt . . . 4
- 2.3 discrete$_{l}og$ . . . 5
- 2.4 euler$_{t}otient$ . . . 5
- 2.5 extended$_{e}uclid$ . . . 5
- 2.6 factorial . . . 5
- 2.7 fft . . . 6
- 2.8 fraction . . . 6
- 2.9 lagrange$_{i}nterpolation$ . . . 7
- 2.10 messel$_{l}ehmer$ . . . 7
- 2.11 mulmat . . . 7
- 2.12 ntt . . . 7
- 2.13 pollard$_{r}ho$ . . . 8
- 2.14 sqrt$_{m}od$ . . . 9

3 misc **9**
- 3.1 4D$_{l}ongestpath$ . . . 9
- 3.2 KSP . . . 10
- 3.3 debug . . . 10
- 3.4 pragma . . . 10

4 string **11**
- 4.1 kmp . . . 11
- 4.2 manacher . . . 11
- 4.3 suffixarray . . . 11
- 4.4 zfunc . . . 12

1 DataStructure

1.1 dsurollback

```
int par[N + 5];
stack<pair<int, int>> his;
void init(int n) {
 while(!his.empty()) his.pop();
 for(int i=1; i<=n; i++) par[i] = -1;
}
int find(int u) {
 return (par[u] < 0 ? u : find(par[u]));
}
bool merge(int u, int v) {
 u = find(u); v = find(v);
 if(u == v) return false;
 if(abs(par[u]) < abs(par[v])) swap(u, v);
 his.push({v, par[v]});
 his.push({u, abs(par[u])});
 par[u] += par[v];
 par[v] = u;
 return true;
}
void rollback() {
 if(his.empty()) return;
 ii u = his.top(); his.pop();
 par[u.fi] = u.se;
 ii v = his.top(); his.pop();
 par[v.fi] = v.se;
}
```

1.2 flow

```
struct FlowEdge {
    int v, u;
    long long cap, flow = 0;
    FlowEdge(int v, int u, long long cap) : v(v), u(u), cap(
        cap) {}
};
struct Dinic {
    const long long flow_inf = 1e18;
    vector<FlowEdge> edges;
    vector<vector<int>> adj;
    int n, m = 0;
    int s, t;
    vector<int> level, ptr;
    queue<int> q;
    Dinic(int n, int s, int t) : n(n), s(s), t(t) {
        adj.resize(n);
        level.resize(n);
        ptr.resize(n);
    }
    void add_edge(int v, int u, long long cap) {
        edges.emplace_back(v, u, cap);
        edges.emplace_back(u, v, 0);
        adj[v].push_back(m);
        adj[u].push_back(m + 1);
        m += 2;
    }
    bool bfs() {
        while (!q.empty()) {
            int v = q.front();
            q.pop();
            for (int id : adj[v]) {
                if (edges[id].cap == edges[id].flow)
                    continue;
                if (level[edges[id].u] != -1)
                    continue;
                level[edges[id].u] = level[v] + 1;
                q.push(edges[id].u);
            }
        }
        return level[t] != -1;
    }
    long long dfs(int v, long long pushed) {
        if (pushed == 0)
            return 0;
        if (v == t)
            return pushed;
        for (int& cid = ptr[v]; cid < (int)adj[v].size(); cid
             ++) {
            int id = adj[v][cid];
            int u = edges[id].u;
            if (level[v] + 1 != level[u])
                continue;
            long long tr = dfs(u, min(pushed, edges[id].cap -
                 edges[id].flow));
            if (tr == 0)
                continue;
            edges[id].flow += tr;
            edges[id ^ 1].flow -= tr;
            return tr;
        }
        return 0;
    }
    long long flow() {
        long long f = 0;
        while (true) {
            fill(level.begin(), level.end(), -1);
            level[s] = 0;
            q.push(s);
            if (!bfs())
                break;
            fill(ptr.begin(), ptr.end(), 0);
            while (long long pushed = dfs(s, flow_inf)) {
                f += pushed;
            }
        }
        return f;
    }
};
```

1.3 fw2d

```
struct Fenwick2D{
    #define gb(x) (x)&(-x)
    vector<vector<int>> nodes,bit;
    vector <int> record,vis;
    int sx;
    void init(int _sx){
        sx = _sx;
        nodes.resize(sx + 1);
        bit.resize(sx + 1);
        vis.resize(sx+1,0);
    }
    void init_nodes(){
        for(int i : record){
            sort(all(nodes[i]));
            nodes[i].resize(unique(all(nodes[i])) - nodes[i].
                 begin());
            bit[i].resize(sz(nodes[i]) + 1);
        }
    }
 void clear(){
  for(auto i:record) nodes[i].clear(),bit[i].clear(),vis[i
      ]=0;
  record.clear();
 }
    void fake_update(int x, int y){
        for(; x <= sx; x += gb(x)){
         if(!vis[x]){
          vis[x]=1;
          record.push_back(x);
         }
            nodes[x].push_back(y);
        }
    }
```

```
    void fake_get(int x, int y){
        for(; x; x -= gb(x)){
         if(!vis[x]){
          vis[x]=1;
          record.push_back(x);
         }
            nodes[x].push_back(y);
        }
    }
    void update(int x, int yy, int val){
        for(; x <= sx; x += gb(x))
            for(int y = lower_bound(all(nodes[x]), yy) -
                 nodes[x].begin() + 1; y <= sz(nodes[x]); y +=
                  gb(y))
                bit[x][y] = max(bit[x][y], val);
    }
    int get(int x, int yy){
        int res = 0;
        for(; x; x -= gb(x))
            for(int y = upper_bound(all(nodes[x]), yy) -
                 nodes[x].begin(); y > 0; y -= gb(y))
                res = max(res, bit[x][y]);
        return res;
    }
}fw;
```

1.4 hld

```
// vi g[N + 5]
int head[N + 5], chain[N + 5];
int sz[N + 5], depth[N + 5], p[N + 5];
int arr[N + 5], pos[N + 5];
int curchain = 1, curpos = 1;
void dfs(int u, int par) {
 sz[u] = 1;
 if(par == -1) depth[u] = 1;
 for(auto v: g[u]) if(v != par) {
  p[v] = u;
  depth[v] = depth[u] + 1;
  dfs(v, u);
  sz[u] += sz[v];
 }
}
void hld(int u, int par) {
 if(!head[curchain]) head[curchain] = u;
 chain[u] = curchain;
 pos[u] = curpos;
 arr[curpos] = u;
 curpos++;
 int nxt = 0;
 for(auto v: g[u]) if(v != par && sz[v] > sz[nxt]) nxt = v;
 if(nxt) hld(nxt, u);
 for(auto v: g[u])
  if(v != par && v != nxt) {curchain++; hld(v, u);}
}
void query(int u, int v) {
 while(chain[u] != chain[v]) {
  if(chain[u] > chain[v]) u = p[head[chain[u]]];
  else v = p[head[chain[v]]];
 }
 if(pos[u] < pos[v]) {}
 else {}
}
```

1.5 lichao

```
const ll INF = 1e18L + 5;
struct implicit_lichao_tree {
    struct node {
        ll a, b;
        ll nxt[2];
        node(ll v1=-INF, ll v2=-INF) {
         a=v1; b=v2;
         nxt[0] = nxt[1] = -1;
        }
        ll operator ()(ll x) {
            if (a == -INF) return -INF;
            return a*x + b;
        }
    };
    vector<node> st;
    void build() {
     st.clear();
     st.pb(node(-INF, -INF));
    }
    void new_node(ll id, ll o) {
     st[id].nxt[o] = st.size();
     st.pb(node(-INF, -INF));
     ll idx = st[id].nxt[o];
     st[idx].a = st[id].a;
     st[idx].b = st[id].b;
    }
    void add(ll id, ll l, ll r, ll a, ll b) {
     if(l == r) {
      if(st[id](l) < a*l + b) {
       st[id].a = a;
       st[id].b = b;
      }
      return;
     }
     int mid = (l + r)/2;
     if(r == l + 1) mid = l;
     if(st[id].a > a) {
      swap(st[id].a, a);
      swap(st[id].b, b);
     }
     if(st[id](mid) < a*mid + b) {
      swap(st[id].a, a);
      swap(st[id].b, b);
      if(st[id].nxt[0] == -1) new_node(id, 0);
      add(st[id].nxt[0], l, mid, a, b);
     }
     else {
      if(st[id].nxt[1] == -1) new_node(id, 1);
      add(st[id].nxt[1], mid + 1, r, a, b);
     }
    }
    ll get(ll id, ll l, ll r, ll pos) {
     if(l == r) return st[id](pos);
     int mid = (l + r)/2;
     if(r == l + 1) mid = l;
     if(pos <= mid) {
      if(st[id].nxt[0] == -1) return st[id](pos);
      return max(st[id](pos), get(st[id].nxt[0], l, mid, pos)
           );
     }
     else {
      if(st[id].nxt[1] == -1) return st[id](pos);
      return max(st[id](pos), get(st[id].nxt[1], mid + 1, r,
           pos));
     }
    }
} lt;
```

1.6 pbds

```
#include <bits/stdc++.h>
#include <ext/pb_ds/assoc_container.hpp>
using namespace std;
using namespace __gnu_pbds;
template<typename T>
using ordered_set = tree<T, null_type, less<T>, rb_tree_tag,
     tree_order_statistics_node_update>;
void solve() {
    int n, q;
    cin >> n >> q;
    vector<int> a(n);
```

```
    for(auto &it: a) {cin >> it;}
    ordered_set<int> me(begin(a), end(a));
    while(q--) {
        int t, x;
        cin >> t >> x;
        if(t == 0) {
            me.insert(x);
        } else if(t == 1) {
            me.erase(x);
        } else if(t == 2) {
            auto it = me.find_by_order(x-1);
            cout << (it == end(me) ? -1 : *it) << '\n';
        } else if(t == 3) {
            cout << me.order_of_key(x+1) << '\n';
        } else if(t == 4) {
            auto it = me.upper_bound(x);
            cout << (it == begin(me) ? -1 : *prev(it)) << '\n
                ';
        } else if(t == 5) {
            auto it = me.lower_bound(x);
            cout << (it == end(me) ? -1 : *it) << '\n';
        }
    }
}
```

2 math

2.1 $_{i}nt128$

```
#include <bits/stdc++.h>
using namespace std;
using i128 = __int128_t;
i128 str2i128(std::string str) {
    i128 ret = 0;
    bool minus = false;
    for (auto c : str) {
        if (c == '-') minus = true;
        else ret = ret * 10 + c - '0';
    }
    return minus ? -ret : ret;
}
istream &operator>>(std::istream &is, i128 &x) {
    string s;
    return is >> s, x = str2i128(s), is;
}
ostream &operator<<(std::ostream &os, const i128 &x) {
    i128 tmp = x;
    if (tmp == 0) return os << 0;
    vector<int> ds;
    if (tmp < 0) {
        os << '-';
        while (tmp) {
            int d = tmp % 10;
            if (d > 0) d -= 10;
            ds.emplace_back(-d), tmp = (tmp - d) / 10;
        }
    } else {
        while (tmp) ds.emplace_back(tmp % 10), tmp /= 10;
    }
    reverse(ds.begin(), ds.end());
    for (auto i : ds) os << i;
    return os;
}
i128 my_abs(i128 n) {
    if (n < 0) return -n;
    return n;
}
i128 gcd(i128 a, i128 b) {
    if (b == 0) return a;
    return gcd(b, a % b);
}
// Count trailing zeroes
int ctz128(i128 n) {
    if (!n) return 128;
    if (!static_cast<uint64_t>(n)) return __builtin_ctzll(
        static_cast<uint64_t>(n >> 64)) + 64;
    else return __builtin_ctzll(static_cast<uint64_t>(n));
}
```

2.2 crt

```
#include "bits/stdc++.h"
using namespace std;
int exgcd(int a, int b, int& x, int& y) {
    if(!b){x=1;y=0;return a;}
    int x1,y1;
    int d=exgcd(b,a%b,x1,y1);
    x=y1;
    y=x1-y1*(a/b);
    return d;
}
int crt(int a1, int m1, int a2, int m2){
 int n1,n2;
 int d=exgcd(m1,m2,n1,n2);
 return (a1*n2*m2+a2*n1*m1+m1*m2*mod)%(m1*m2);
}
int crt(vector <pair <int,int> > &m){
 if((int)m.size()==1){
  if(!m[0].first) return m[0].second;
  return m[0].first;
 }
 int ans=crt(m[0].first,m[0].second,m[1].first,m[1].second);
 int prod=m[0].second*m[1].second;
 for(int i=2; i<(int)m.size(); i++) ans=crt(ans,prod,m[i].
      first,m[i].second),prod*=m[i].second;
 if(!ans) return prod;
 return ans;
}
vector <pair <int,int> > nw;
vector <int> v; // prime numbers 1 -> sqrt
map <int,int> mp;
void fac(int x){
 for(auto p:v){
  if(x==1 || x<p) break;
  if(x%p==0){
   int cnt=1;
   while(x%p==0) x/=p,cnt*=p;
   mp[p]=max(mp[p],cnt);
  }
 }
 if(x>1 and !mp.count(x)) mp[x]=x;
}
void discrete(vector <pair <int,int> > &modu){ // non-prime
     : discrete to diffrent prime factors
 map <int,int> ck;
 for(auto [A,M]:modu){
  for(auto p:v){
   if(M==1 || M<p) break;
   if(M%p==0){
    int mpp=mp[p];
    if(M%mpp==0 and !ck.count(p)){
     ck[p]=1;
     nw.push_back({A%mpp,mpp});
     M/=mpp;
    }else while(M%p==0) M/=p;
   }
  }
  if(M>1){
   if(!ck.count(M) and mp[M]==M){
    ck[M]=1;
    nw.push_back({A%M,M});
   }
  }
 }
}
```

2.3 discrete$_{l}$og

```
// smallest x such that a**x = n (mod p) (1e9 contrainst)
int solve(int a, int n, int p){
 if(!n){
  int pr=a,cnt=1;
  while(1){
   if(cnt>=30) return -1;
   if(!pr) return cnt;
   pr=(pr*a)%p;
   cnt++;
  }
 }
 int g=__gcd(a,p),A=a;
 if(n%g) return -1;
 a/=g;n/=g;p/=g;A%=p;
 if(a==n) return 1;
 if(!A) return -1;
 if(n%__gcd(A,p)) return -1;
 unordered_map <int,int> mp;
 int s=sqrt(p),x=1,prod=a,ans=INT_MAX;;
 if(s*s==p) s--;
 for(int i=1; i<=s; i++) x=(x*A)%p;
 for(int i=1; i<=s+1; i++){
  prod=(prod*x)%p;
  if(!mp.count(prod)) mp[prod]=i;
 }
 prod=n;
 for(int i=0; i<=s; i++){
  if(mp.count(prod)) ans=min(ans,mp[prod]*s-i);
  prod=(prod*A)%p;
 }
 if(ans==INT_MAX || a*powmod(A,ans)%p!=n) ans=-2;
 return ans+1;
}
```

2.4 euler$_{t}$otient

```
#include <bits/stdc++.h>
using namespace std;
const int mod=998244353;const int N=2e6;const int inv2
    =499122177;
template<int N>
struct Sieve{
private:
 int phi[N];
 vector <int> primes;
public:
 constexpr Sieve() {
  memset(phi, 0, sizeof phi);
  for (int i = 2; i < N; i++) {
   if (phi[i] == 0) {
    phi[i] = i;
    primes.push_back(i);
   }
   for (int j = 0; primes[j] * i < N; ++j) {
    phi[i * primes[j]] = primes[j];
    if (primes[j] == phi[i]) break;
   }
  }
  phi[1] = 1;
  for (int i = 2; i < N; i++) {
   int j = phi[i];
   if ((i / j) % j == 0) phi[i] = phi[i / j] * j;
   else phi[i] = phi[i / j] * (j - 1);
  }
  for (int i = 1; i < N; i++){
    phi[i] += phi[i-1];
    if(phi[i]>=mod) phi[i]-=mod;
  }
 }
 inline constexpr int operator[](std::size_t index) const
 {
  return phi[index];
 }
};
Sieve<N> phi;
unordered_map <long long,int> mp;
inline int sq(long long n) {
 return round(sqrt(n));
}
inline int rphi(long long n) {
 return (phi[n] - phi[n - 1] + mod) % mod;
}
int sum(long long n) {
 if (n < N) return phi[n];
 if (mp.count(n)) return mp[n];
 int y = sq(n);
 int x = sum(n / (y + 1));
 int pr = ((((n % mod) * ((n + 1) % mod)) % mod) * inv2) %
      mod;
 for (int i = 1; n / i > y; i++) {
  pr -= ((long long)rphi(i) * (n / i)) % mod;
  if (pr < 0) pr += mod;
 }
 for (int j = y; j > 1; j--) {
  int p = sum(n / j);
  pr -= ((long long)(p - x + mod) * j) % mod;
  if (pr < 0) pr += mod;
  x = p;
 }
 int val=pr+x;
 if(val>=mod) val-=mod;
 return mp[n]=val;
}
long long n;
signed main(){
 mp.max_load_factor(0.25);
 cin>>n;
 cout<<sum(n);
}
```

2.5 extended$_{e}$uclid

```
// ax+by=d
// ton tai n0 khi d chia het gcd(a,b)
template<typename T>
T extgcd(T a, T b, T &x, T &y) {
    T g = a; x = 1; y = 0;
    if (b != 0) g = extgcd(b, a % b, y, x), y -= (a / b) * x;
    return g;
}
```

2.6 factorial

```
// factorial up to 1e9
__attribute__((target("avx2"), optimize("O3,unroll-loops")))
      int calc(int n){
  if (n >= mod) return 0;
  if (n * 2 > mod) {
    int f = calc(mod - 1 - n);
    if (n % 2 == 0) f = mod - f;
    int res = 1;
    for (int e = mod - 2; e; f = 1LL * f * f % mod, e >>= 1)
          if (e & 1) res = 1LL * res * f % mod;
    return res;
  }
  int a[8] = {1, 1, 1, 1, 1, 1, 1, 1};
  int b[8] = {1, 2, 3, 4, 5, 6, 7, 8};
  for (int i = 1; i <= n / 8; i++) {
    for (int j = 0; j < 8; j++) a[j] = 1LL * a[j] * b[j] %
         mod;
    for (int j = 0; j < 8; j++) b[j] += 8;
  }
  long long ans = 1;
  for (int j = 0; j < 8; j++) ans = ans * a[j] % mod;
```

```
    for (int x = n / 8 * 8 + 1; x <= n; x++) ans = ans * x %
        mod;
    return ans;
}
```

2.7 fft

```
#include "bits/stdc++.h"
using namespace std;

const int LG=20;
const int N=1<<LG;
const double PI=acos(-1);

struct Complex{
    double r,i;
    Complex() : r(0), i(0) {}
    Complex(double t) : r(cos(t)), i(sin(t)) {}
    Complex(double r, double i) : r(r), i(i) {}
    const Complex operator + (const Complex &o) const {
        return Complex(r+o.r,i+o.i);
    }
    const Complex operator - (const Complex &o) const {
        return Complex(r-o.r,i-o.i);
    }
    const Complex operator * (const Complex &o) const {
        return Complex(r*o.r-i*o.i,r*o.i+i*o.r);
    }
    void operator /= (double d){
        this->r/=d;
        this->i/=d;
    }
};
int reverse(int mask){
    int r=0;
    for(int i=0; i<LG; i++) r|=((mask&(1<<i))>>i)<<(LG-i-1);
    return r;
}
void fft(vector<Complex> &a, bool inv=0){
    vector<Complex> b=a;
    for(int i=0; i<N; i++) b[i]=a[reverse(i)];
    for(int iter=0; iter<LG; iter++){
        int len=(1<<iter);
        Complex w((inv ? 1 : -1) * PI / len);
        int i=0,j=0;
        for(int cnt=(N>>(iter+1)); cnt--; ){
            i=j;
            j=i+len;
            Complex t(1.0,0.0);
            for(int l=len; l--; ){
                a[i]=b[i]+t*b[j];
                a[j]=b[i]-t*b[j];
                t=t*w;
                i++;
                j++;
            }
        }
        swap(a,b);
    }
    a=b;
    b=vector<Complex>();
    if(inv) for(int i=0; i<N; i++) a[i]/=N;
}
signed main(){
 ios_base::sync_with_stdio(false);
    cin.tie(NULL);
    vector <Complex> a(N),b(N),c(N);
    fft(a);
 fft(b);
 for(int i=0; i<N; i++) c[i]=a[i]*b[i];
 fft(c,1);
 for(int i=0; i<N; i++) cout<<(int)(c[i].r+0.5)<<"\n";
    return 0;
}
```

2.8 fraction

```
#include <bits/stdc++.h>
using namespace std;
int cmp(int a, int b) { return (a == b) ? 0 : ((a < b) ? -1
    : 1); }
struct Fraction {
    int a, b;
    Fraction() {
        a = 0; b = 1;
    }
    Fraction(int _a, int _b) {
        a = _a;
        b = _b;
        norm();
    }
    Fraction(int x) {
        a = x; b = 1;
    }
    Fraction operator + (const Fraction& other) const {
        return Fraction(a * other.b + b * other.a, b * other.
            b);
    }
    Fraction operator - (const Fraction& other) const {
        return Fraction(a * other.b - b * other.a, b * other.
            b);
    }
    Fraction operator * (const Fraction& other) const {
        return Fraction(a * other.a, b * other.b);
    }
    Fraction operator / (const Fraction& other) const {
        assert(other.a != 0);
        return Fraction(a * other.b, b * other.a);
    }
    int cmp(Fraction other) const {
        return ::cmp(a * other.b, b * other.a);
    }

#define Comp(x) bool operator x (Fraction q) const { return
     cmp(q) x 0; }
    Comp(>) Comp(<) Comp(==) Comp(>=) Comp(<=) Comp(!=)
#undef Comp
    void norm() {
        if (b < 0) a = -a,b = -b;
        if (a == 0) b = 1;
        else {
            int g = __gcd(llabs(a), llabs(b));
            a /= g,b /= g;
        }
    }
};
istream& operator >> (istream& cin, Fraction& p) {
    cin >> p.a >> p.b;
    return cin;
}
ostream& operator << (ostream& cout, Fraction& p) {
    cout << p.a << '/' << p.b;
    return cout;
}
signed main(){
 ios_base::sync_with_stdio(0);
 cin.tie(0);
 Fraction ans;
 int n;
 cin>>n;
 for(int i=1; i<=n; i++){
  Fraction a;
  cin>>a;
  ans=ans+a;
 }
 cout<<ans;
 return 0;
}
```

2.9 lagrange_interpolation

```
// f(0)->f(n) : find f(x)
int lagrange_interpolate(int n, int x){
 vector <int> pre(n+2),suf(n+2),invfac(n+2);
 pre[0]=x;
 for(int i=1; i<=n; i++) pre[i]=(pre[i-1]*(x-i))%mod;
 suf[n+1]=1;
 for(int i=n; i>=0; i--) suf[i]=(suf[i+1]*(x-i))%mod;
 invfac[0]=1;
 for(int i=1; i<=n; i++) invfac[i]=(invfac[i-1]*powmod(i,mod
     -2,mod))%mod;
 int ans=0;
 for(int i=0; i<=n; i++){
  int res;
  if(!i) res=g[i]*suf[i+1]%mod*invfac[n]%mod;
  else res=g[i]*pre[i-1]%mod*suf[i+1]%mod*invfac[i]%mod*
       invfac[n-i]%mod;
  if((n-i+1)&1) ans=(ans+res)%mod;
  else ans=(ans-res+mod)%mod;
 }
 return ans;
}
```

2.10 messel_lehmer

```
#include <bits/stdc++.h>
using namespace std;
/// Magic constants, optimized to answer prime counting
     queries for n=10^12 but can be tweaked
const int MAXV = 20000010;const int MAXP = 7;const int MAXN
     = 50;const int MAXM = 2 * 3 * 7 * 5 * 11 * 13 * 17;
constexpr auto fast_div = [](const long long& a, const int&
     b) ->long long {return double(a) / b + 1e-9;};
vector<int> primes;
bitset<MAXV> is_prime;
int prod[MAXP], pi[MAXV], dp[MAXN][MAXM];
void sieve(){
    is_prime[2] = 1;
    for (int i = 3; i < MAXV; i += 2) is_prime[i] = 1;
    for (int i = 3; i * i < MAXV; i += 2) for (int j = i * i;
          is_prime[i] && j < MAXV; j += (i << 1)) is_prime[j]
          = 0;
    for (int i = 1; i < MAXV; i++){
        pi[i] = pi[i - 1] + is_prime[i];
        if (is_prime[i]) primes.push_back(i);
    }
}
void gen(){
    int i, j;
    sieve();
    for (prod[0] = primes[0], i = 1; i < MAXP; i++) prod[i] =
          prod[i - 1] * primes[i];
    for (i = 0; i < MAXM; i++) dp[0][i] = i;
    for (i = 1; i < MAXN; i++) for (j = 1; j < MAXM; j++) dp[
         i][j] = dp[i - 1][j] - dp[i - 1][fast_div(j, primes[
         i - 1])];
}
uint64_t phi(long long m, int n){
    if (!n) return m;
    if (n < MAXN && m < MAXM) return dp[n][m];
    if (n < MAXP) return dp[n][m % prod[n - 1]] + fast_div(m,
          prod[n - 1]) * dp[n][prod[n - 1]];
    long long p = primes[n - 1];
    if (m < MAXV && p * p >= m) return pi[m] - n + 1;
    if (p * p * p < m || m >= MAXV) return phi(m, n - 1) -
         phi(fast_div(m, p), n - 1);
    int lim = pi[(int)sqrt(0.5 + m)];
    uint64_t res = pi[m] - (lim + n - 2) * (lim - n + 1) / 2;
    for (int i = n; i < lim; i++) res += pi[fast_div(m,
         primes[i])];
    return res;
}
uint64_t messel_lehmer(long long n){
    if (n < MAXV) return pi[n];
    int s = sqrt(0.5 + n), c = cbrt(0.5 + n);
    uint64_t res = phi(n, pi[c]) + pi[c] - 1;
    for (int i = pi[c]; i < pi[s]; i++) res -= messel_lehmer(
         fast_div(n, primes[i])) - i;
    return res;
}
long long n;
signed main(){
    gen();
    cin>>n;
    cout<<messel_lehmer(n);
}
```

2.11 mulmat

```
struct Matrix{
    vector<vector<int>> mat;
    int n,m;
    Matrix(int _n, int _m){
        n=_n;
        m=_m;
        mat.resize(n, vector<int>(m, 0));
    }
    Matrix identity(int _n){
        Matrix a=Matrix(_n,_n);
        while(_n--) a.mat[_n][_n]=1;
        return a;
    }
    Matrix operator * (const Matrix &b){
        Matrix a=*this;
        assert(a.m==b.n);
        Matrix res(a.n,b.m);
        for(int i=0; i<a.n; i++){
            for(int j=0; j<b.m; j++){
                for(int k=0; k<a.m; k++){
                    res.mat[i][j]+=a.mat[i][k]*b.mat[k][j]%mod
                         ;
                    res.mat[i][j]%=mod;
                }
            }
        }
        return res;
    }
    Matrix pow(int exp){
        assert(n==m);
        Matrix base=*this,ans=identity(n);
        for(; exp>0; exp>>=1ll, base=base*base) if(exp&1) ans
             = ans*base;
        return ans;
    }
};
```

2.12 ntt

```
const ll mod_ntt = 998244353;
const ll root = 15311432, root_1 = 469870224, root_pw = 1LL
     <<23;
ll powmod(ll a, ll b) {
    ll ans = 1;
    while(b) {
     if(b & 1) ans = ans * a % mod_ntt;
     a = a * a % mod_ntt;
     b /= 2;
    }
    return ans;
}
void ntt(vector<long long>& a, bool inv) {
    ll n = a.size();
    for (ll i = 1, j = 0; i < n; i++) {
        ll bit = (n>>1);
        for (; j&bit; bit>>=1) {j ^= bit;} j ^= bit;
        if (i < j) swap(a[i],a[j]);
```

```
    }
    for (ll l = 2; l <= n; l <<= 1) {
        ll wl = inv ? root_1 : root;
        for (ll i = l; i < root_pw; i <<= 1) wl = ((wl*wl)%
             mod_ntt);
        for (ll i = 0; i < n; i += l) {
            ll w = 1;
            for (ll j = 0; j < l/2; j++) {
                ll u = a[i+j], v = ((a[i+j+l/2]*w)%mod_ntt);
                a[i+j] = ((u+v) < mod_ntt) ? (u+v) : (u+v-
                     mod_ntt);
                a[i+j+l/2] = ((u-v) >= 0) ? (u-v) : (u-v+
                     mod_ntt);
                w = (w*wl)%mod_ntt;
            }
        }
    }
    if (inv) {
        ll n_1 = powmod(n,mod_ntt-2);
        for (ll& i : a) i = (i*n_1)%mod_ntt;
    }
}
vector<long long> mul(vector<long long> const& a, vector<
     long long> const& b) {
    vector<long long> fa(a.begin(),a.end());
    vector<long long> fb(b.begin(),b.end());
    ll n = 1;
    while(n < (int)a.size() + (int)b.size()) n <<= 1;
    fa.resize(n); fb.resize(n);
    ntt(fa,0); ntt(fb,0);
    for(ll i=0; i<n; i++) fa[i] = (fa[i]*fb[i])%mod_ntt;
    ntt(fa,1);
    return fa;
}
```

2.13 pollard$_r$$ho$

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
using ull = unsigned long long;
using ld = long double;
mt19937_64 rng(chrono::steady_clock::now().time_since_epoch
     ().count());
ll rand(ll x){
 return (ull)(rng())%x;
}
ll modmul(ll a, ll b, ll M){
 ll ret=a*b-M*(ll)(1.L/M*a*b);
 return ret+M*(ret<0)-M*(ret>=(ll)M);
}
ll modpow(ll b, ll e, ll mod){
 ll ans=1;
 for (; e; b=modmul(b,b,mod), e/=2) if(e&1) ans=modmul(ans,b
      ,mod);
 return ans;
}
bool isPrime(ll n){
 if(n<2 || n%6%4!=1) return (n|1)==3;
 ll A[]={2,325,9375,28178,450775,9780504,1795265022},s=
      __builtin_ctzll(n-1), d=n>>s;
 for(ll a:A){
  ll p=modpow(a%n,d,n),i=s;
  while(p!=1 && p!=n-1 && a%n && i--) p=modmul(p,p,n);
  if (p!=n-1 && i!=s) return 0;
 }
 return 1;
}
ll gcd(ll x, ll y) {
    return y == 0 ? x : gcd(y, x % y);
}
void pollard(ll x, vector<ll> &ans) {
    if(isPrime(x)){
        ans.push_back(x);
        return;
    }
    ll c=1;
    while(1){
        c=1+rand(x-1);
        auto f = [&](ll y) {
            ll res=modmul(y,y,x)+c;
            if(res>=x) res-=x;
            return res;
        };
        int B=100,len=1;
        ll y=2,g=1;
        while(g==1){
            ll z=y,zs=-1;
            for(int i=0; i<len; i++) z=f(z);
            int lft=len;
            while(g==1 and lft>0){
                zs=z;
                ll p=1;
                for(int i=0; i<B and i<lft; i++){
                    p=modmul(p,abs(z-y),x);
                    z=f(z);
                }
                g=gcd(p,x);
                lft-=B;
            }
            if(g==1){
                y=z;
                len<<=1;
                continue;
            }
            if(g==x){
                g=1;
                z=zs;
                while(g==1){
                    g=gcd(abs(z-y),x);
                    z=f(z);
                }
            }
            if(g==x) break;
            pollard(g,ans);
            pollard(x/g,ans);
            return;
        }
    }
}
vector<ll> factorize(ll x) {
    vector<ll> ans;
    for (ll p:{2,3,5,7,11,13,17,19}){
        while(x%p==0){
            x/=p;
            ans.push_back(p);
        }
    }
    if(x!=1) pollard(x, ans);
    sort(ans.begin(),ans.end());
    return ans;
}
vector< pair<ll, int> > factorize_pk(ll x){
 auto ps=factorize(x);
 ll last=-1,cnt=0;
 vector< pair<ll, int> > res;
  for(auto p:ps){
    if(p==last) cnt++;
    else{
        if(~last) res.emplace_back(last,cnt);
        last=p;
        cnt=1;
    }
 }
 if(cnt) res.emplace_back(last, cnt);
 return res;
}
vector<ll> get_all_divisors(ll n){
 auto pks=factorize_pk(n);
```

```
 vector<ll> res;
 function<void(int, ll)> gen=[&](int i, ll prod){
    if(i==static_cast<int>(pks.size())) {
        res.push_back(prod);
        return;
    }
    ll cur_power=1;
    for(int cur=0; cur<=pks[i].second; cur++){
        gen(i+1, prod*cur_power);
        cur_power*=pks[i].first;
    }
 };
 gen(0,1LL);
 sort(res.begin(), res.end());
 return res;
}
```

2.14 sqrt$_{m}od$

```
#include <bits/stdc++.h>
using namespace std;
#define ll long long
mt19937_64 rng(chrono::steady_clock::now().time_since_epoch
    ().count());
ll rand(ll x){
 return (ll)(rng())%x;
}
ll euclid(ll x, ll y, ll &k, ll &l) {
    if (y == 0) {
        k = 1; l = 0;
        return x;
    }
    ll g = euclid(y, x % y, l, k);
    l -= k * (x / y);
    return g;
}
ll mult(ll x, ll y, ll md) {
    return x * y % md;
}
ll bin_pow(ll x, ll p, ll md) {
    if (p == 0) return 1;
    if (p & 1) return mult(x, bin_pow(x, p - 1, md), md);
    return bin_pow(mult(x, x, md), p / 2, md);
}
ll Cipolla(ll X, ll q) {
    ll pw = (q - 1) / 2;
    int K = 60;
    while((1LL << K) > pw) K--;
    while(1){
        ll t = rand(q), a = 0, b = 0, c = 1;
        for (int k = K; k >= 0; k--) {
            a = b * b % q;
            b = 2 * b * c % q;
            c = (c * c + a * X) % q;
            if (!((pw >> k) & 1)) continue;
            a = b;
            b = (b * t + c) % q;
            c = (c * t + a * X) % q;
        }
        if (b == 0) continue;
        c = (c + q - 1) % q;
        ll k, l;
        euclid(b, q, k, l);
        c = -c * k % q;
        if (c < 0) c += q;
        if (c * c % q == X) return c;
    }
    assert(0);
}
// find a such that a^2 dd X mod q
ll sqrtMod(ll X, ll q) {
    X %= q;
    if (q == 2 || X == 0) return min(X, q-X);
    if (bin_pow(X, (q - 1) / 2, q) != 1) return -1;
    if (q % 4 == 3) {
        ll res = bin_pow(X, (q + 1) / 4, q);
        return min(res, q - res);
    }
    auto res = (Cipolla(X, q) % q + q) % q;
    return min(res, q-res);
}
```

3 misc

3.1 4D$_{l}ongestpath$

```
// longest path such that (i->j) <-> (ai<=aj,bi<=bj,ci<=cj,
    di<=dj)
#include "bits/stdc++.h"
using namespace std;
#define ll long long
#define sz(x) (int)((x).size())
#define all(x) (x).begin(), (x).end()
const int N=5e4+5;
const int mod=1e9+7;
int n,dp[N];
vector < tuple <int,int,int,int> > mahiru;
vector <int> compress;
struct Data{
 int x,y,z,id;
};
vector <Data> v;
// fw2d
void cdq(int l, int r){
 if(l+1==r) return;
 int mid=(l+r)>>1;
 cdq(l,mid);
 vector <Data> tmp,cecilia,fake;
 for(int i=mid; i<r; i++) fake.push_back(v[i]);
 sort(v.begin()+mid,v.begin()+r,[&](Data a, Data b){
  return a.x<b.x;
 });
 for(int i=l; i<mid; i++) fw.fake_update(v[i].y,v[i].z);
 for(int i=mid; i<r; i++) fw.fake_get(v[i].y,v[i].z);
 fw.init_nodes();
 int a=l,b=mid;
 while(a<mid and b<r){
  if(v[a].x<=v[b].x) fw.update(v[a].y,v[a].z,dp[v[a].id]),
       tmp.push_back(v[a]),cecilia.push_back(v[a++]);
  else dp[v[b].id]=max(dp[v[b].id],fw.get(v[b].y,v[b].z)+1),
       cecilia.push_back(v[b++]);
 }
 while(a<mid) tmp.push_back(v[a]),cecilia.push_back(v[a++]);
 while(b<r) dp[v[b].id]=max(dp[v[b].id],fw.get(v[b].y,v[b].z
     )+1),cecilia.push_back(v[b++]);
 for(int i=l; i<mid; i++) v[i]=tmp[i-l];
 for(int i=mid; i<r; i++) v[i]=fake[i-mid];
 fw.clear();
 vector <Data> ().swap(tmp);
 cdq(mid,r);
 for(int i=l; i<r; i++) v[i]=cecilia[i-l];
 vector <Data> ().swap(cecilia);
}
signed main(){
 ios_base::sync_with_stdio(false);
    cin.tie(NULL);
    fw.init(N+N);
 cin>>n;
 for(int i=1; i<=n; i++){
  dp[i]=1;
  int x,y,z,t;
  cin>>x>>y>>z>>t;
  compress.push_back(z);
  compress.push_back(t);
  mahiru.push_back({x,y,z,t});
 }
 sort(compress.begin(),compress.end());
```

```
 compress.erase(unique(compress.begin(),compress.end()),
      compress.end());
 sort(mahiru.begin(),mahiru.end(),[&](tuple <int,int,int,int
      > a, tuple <int,int,int,int> b){
  if(get<0>(a)==get<0>(b)){
   if(get<1>(a)==get<1>(b)){
    if(get<2>(a)==get<2>(b)) return get<3>(a)<get<3>(b);
    return get<2>(a)<get<2>(b);
   }
   return get<1>(a)<get<1>(b);
  }
  return get<0>(a)<get<0>(b);
 });
 for(auto [i,x,y,z]:mahiru) v.push_back({x,y,z,(int)v.size()
      +1});
 for(auto &[x,y,z,id]:v){
  y=lower_bound(compress.begin(),compress.end(),y)-compress.
       begin()+1;
  z=lower_bound(compress.begin(),compress.end(),z)-compress.
       begin()+1;
 }
 cdq(0,n);
 cout<<*max_element(dp+1,dp+n+1);
    return 0;
}
```

3.2 KSP

```
//  Knights  shortest path (from (0, 0))
int KSP(int x, int y) {
    if(x<0) x=-x;
    if(y<0) y=-y;
    if(x<y) swap(x,y);
    if(x==1 && y==0) return 3;
    if(x==2 && y==2) return 4;
    int d=x-y;
    if(y>d) return 2*((y-d+2)/3)+d;
    return d-2*((d-y)/4);
}
```

3.3 debug

```
#include <bits/stdc++.h>
#ifndef DEBUG_H
#define DEBUG_H
using namespace std;
// Define __print for various data types
void __print(int x) { cerr << x; }
void __print(long x) { cerr << x; }
void __print(long long x) { cerr << x; }
void __print(unsigned x) { cerr << x; }
void __print(unsigned long x) { cerr << x; }
void __print(unsigned long long x) { cerr << x; }
void __print(float x) { cerr << x; }
void __print(double x) { cerr << x; }
void __print(long double x) { cerr << x; }
void __print(char x) { cerr << '\'' << x << '\''; }
void __print(const char *x) { cerr << '\"' << x << '\"'; }
void __print(const string &x) { cerr << '\"' << x << '\"'; }
void __print(bool x) { cerr << (x ? "true" : "false"); }
template<typename T, typename V>
void __print(const pair<T, V> &x) { cerr << '{'; __print(x.
     first); cerr << ','; __print(x.second); cerr << '}'; }
template<typename T>
void __print(const T &x) { int f = 0; cerr << '{'; for (auto
      &i: x) cerr << (f++ ? "," : ""), __print(i); cerr << "
     }"; }
void _print() { cerr << "\n"; }
template <typename T, typename... V>
void _print(T t, V... v) { __print(t); if (sizeof...(v))
     cerr << " "; _print(v...); }

#ifndef ONLINE_JUDGE
#define cebug(x...) do { cerr << "[" << #x << "] = ";
     _print_debug(x); } while (0)
#else
#define cebug(x...)
#endif

template <typename... Args>
void _print_debug(Args... args);

template <typename T>
void _print_debug_helper(const T &t) {
    __print(t);
}
template <typename T, typename... Args>
void _print_debug(T t, Args... args) {
    _print_debug_helper(t);
    if (sizeof...(args)) {
        cerr << " ";
        _print_debug(args...);
    } else {
        _print();
    }
}
#endif // DEBUG_H
```

```
signed main(){
 ios_base::sync_with_stdio(0);
 cin.tie(0);
 vector <int> v={1,5,2,4,3};
 set <int> s={1,2,3,4};
 pair <int,int> p={1,2};
 cebug(p);cebug(v);cebug(s);
 return 0;
}
```

3.4 pragma

```
#pragma GCC optimize("Ofast,unroll-loops")
#pragma GCC target("arch=skylake")

#pragma GCC optimize("Ofast,unroll-loops")
#pragma GCC target("avx2,fma,bmi,bmi2,popcnt,lzcnt,tune=
     native")

#pragma GCC optimize("Ofast")
#pragma GCC optimization("unroll-loops, no-stack-protector")
#pragma GCC target("avx,avx2,fma")

#pragma GCC ivdep

__attribute__((always_inline)) void smin(int &a, int b) {a =
      a < b ? a : b;}
__attribute__((always_inline)) void smax(int &a, int b) {a =
      a > b ? a : b;}
#define _F __attribute__((always_inline))
_F int add(int a, int b) {return (a+=b)>=MOD?a-=MOD:a;}
_F int sub(int a, int b) {return (a-=b)<0?a+MOD:a;}
_F int mul(int a, int b) {return (a*1LL*b)%MOD;}
_F void set_add(int& a, int b) {if((a+=b)>=MOD) a-=MOD;}
_F void set_sub(int& a, int b) {if((a-=b)<0) a+=MOD;}
_F void set_mul(int& a, int b) {a=mul(a, b);}
_F int exp(int a, int b) {
    int ans=1;
    for(; b; b>>=1) {
        if(b&1) set_mul(ans, a);
        set_mul(a, a);
    }
    return ans;
}
#undef _F
int read(){
 int res=0;bool bo=false;char c;
 while (((c=getchar())<'0'||c>'9')&&c!='-');
 if (c=='-') bo=true;
```

```
 else res=c-'0';
 while ((c=getchar())>='0'&&c<='9') res=(res<<3)+(res<<1)+c-
     '0';
 return bo?~res+1:res;
}
mt19937_64 rng(chrono::steady_clock::now().time_since_epoch
    ().count());
int rand(int l, int r){
    assert(l<=r);
    return uniform_int_distribution<int> (l,r)(rng);
}
#define IO_FAST 1

const int BufSize=1<<21;
struct FastIO
{
 char ibuf[BufSize+10],*p,*e;
 char obuf[BufSize+10],*q,stk[128];

#if IO_FAST
 FastIO(){q=obuf;}
 ~FastIO(){fwrite(obuf,1,q-obuf,stdout);}

 int getChar()
 {
  if(p==e)p=ibuf,e=ibuf+fread(ibuf,1,BufSize,stdin);
  return p==e?EOF:*p++;
 }
 void putChar(int c)
 {
  if(q-obuf==BufSize)fwrite(obuf,1,BufSize,stdout),q=obuf;
  *q++=c;
 }
#else
 int getChar(){return getchar();}
 void putChar(int c){putchar(c);}
#endif

 FastIO& operator>>(char& c)
 {
  do{c=getChar();}while(isspace(c));
  return *this;
 }
 FastIO& operator>>(char* s)
 {
  *s=' ';
  while(isspace(*s))*s=getChar();
  while(!isspace(*s))*++s=getChar();
  *s=0;
  return *this;
 }
 template<typename T>
 FastIO& operator>>(T& x)
 {
  char c,l;
  for(c=0;!isdigit(c);c=getChar())l=c;
  for(x=0;isdigit(c);c=getChar())x=x*10-'0'+c;
  if(l=='-')x=-x;
  return *this;
 }

 FastIO& operator<<(char c)
 {
  putChar(c);
  return *this;
 }
 FastIO& operator<<(char* s)
 {
  while(*s)putChar(*s++);
  return *this;
 }
 FastIO& operator<<(const char* s)
 {
  while(*s)putChar(*s++);
  return *this;
 }
 template<typename T>
 FastIO& operator<<(T x)
 {
  int tp=0;
  if(x<0)x=-x,putChar('-');
  do{stk[++tp]=x%10,x/=10;}while(x);
  while(tp)putChar(stk[tp--]+'0');
  return *this;
 }
}IO;
// IO>>n, IO<<ans
// stress
: 100

gen.exe
brute.exe
stress.exe

fc /w arya.ans arya.out

if errorlevel 1 (exit)

goto 100
```

4 string

4.1 kmp

```
// pi[i] : longest substring being both prefix and suffix
vector <int> prefix_func(vector<int> s){
 int n=(int)s.size();
 vector <int> pi(n,0);
 for(int i=1; i<n; i++){
  int j=pi[i-1];
  while(j and s[i]!=s[j]) j=pi[j-1];
  if(s[i]==s[j]) pi[i]=++j;
 }
 return pi;
}
```

4.2 manacher

```
vector<int> manacher(const string &s){
   int n=(int)s.size();
   vector<int> res(2 * n - 1, 0);
   int l = -1, r = -1;
   for (int z = 0; z < 2 * n - 1; z++) {
     int i = (z + 1) >> 1;
     int j = z >> 1;
     int p = (i >= r ? 0 : min(r - i, res[2 * (l + r) - z]));
     while (j + p + 1 < n && i - p - 1 >= 0) {
        if (!(s[j + p + 1] == s[i - p - 1])) break;
        p++;
     }
     if (j + p > r) {
        l = i - p;
        r = j + p;
     }
     res[z] = p;
   }
   return res;
}
```

4.3 suffixarray

```
#include "bits/stdc++.h"
using namespace std;
template <typename T>
vector<int> suffix_array(int n, const T &s, int char_bound){
  vector<int> a(n);
```

```
if (n == 0) {
  return a;
}
if (char_bound != -1) {
  vector<int> aux(char_bound, 0);
  for (int i = 0; i < n; i++) {
    aux[s[i]]++;
  }
  int sum = 0;
  for (int i = 0; i < char_bound; i++) {
    int add = aux[i];
    aux[i] = sum;
    sum += add;
  }
  for (int i = 0; i < n; i++) {
    a[aux[s[i]]++] = i;
  }
} else {
  iota(a.begin(), a.end(), 0);
  sort(a.begin(), a.end(), [&s](int i, int j) { return s[i]
       < s[j]; });
}
vector<int> sorted_by_second(n);
vector<int> ptr_group(n);
vector<int> new_group(n);
vector<int> group(n);
group[a[0]] = 0;
for (int i = 1; i < n; i++) {
  group[a[i]] = group[a[i - 1]] + (!(s[a[i]] == s[a[i -
      1]]));
}
int cnt = group[a[n - 1]] + 1;
int step = 1;
while (cnt < n) {
  int at = 0;
  for (int i = n - step; i < n; i++) {
    sorted_by_second[at++] = i;
  }
  for (int i = 0; i < n; i++) {
    if (a[i] - step >= 0) {
      sorted_by_second[at++] = a[i] - step;
    }
  }
  for (int i = n - 1; i >= 0; i--) {
    ptr_group[group[a[i]]] = i;
  }
  for (int i = 0; i < n; i++) {
    int x = sorted_by_second[i];
    a[ptr_group[group[x]]++] = x;
  }
  new_group[a[0]] = 0;
  for (int i = 1; i < n; i++) {
    if (group[a[i]] != group[a[i - 1]]) {
      new_group[a[i]] = new_group[a[i - 1]] + 1;
    } else {
      int pre = (a[i - 1] + step >= n ? -1 : group[a[i - 1]
          + step]);
      int cur = (a[i] + step >= n ? -1 : group[a[i] + step
          ]);
      new_group[a[i]] = new_group[a[i - 1]] + (pre != cur);
    }
  }
  swap(group, new_group);
  cnt = group[a[n - 1]] + 1;
  step <<= 1;
 }
 return a;
}
template <typename T>
vector<int> suffix_array(const T &s, int char_bound) {
 return suffix_array((int) s.size(), s, char_bound);
}
template <typename T>
vector<int> build_lcp(int n, const T &s, const vector<int> &
    sa) {
 assert((int) sa.size() == n);
 vector<int> pos(n);
 for (int i = 0; i < n; i++) {
   pos[sa[i]] = i;
 }
 vector<int> lcp(max(n - 1, 0));
 int k = 0;
 for (int i = 0; i < n; i++) {
   k = max(k - 1, 0);
   if (pos[i] == n - 1) {
     k = 0;
   } else {
     int j = sa[pos[i] + 1];
     while (i + k < n && j + k < n && s[i + k] == s[j + k])
         {
       k++;
     }
     lcp[pos[i]] = k;
   }
 }
 return lcp;
}
template <typename T>
vector<int> build_lcp(const T &s, const vector<int> &sa) {
 return build_lcp((int) s.size(), s, sa);
}
int n;
string s;
signed main(){
 n=(int)s.size();
 vector <int> sa=suffix_array(s,256);
 vector <int> lcp=build_lcp(s,sa);
}
```

4.4 zfunc

```
// z[i] : 0->z[i]-1 = i -> i+z[i]-1
vector <int> z_func(string s){
 int n=(int)s.size();
 vector <int> z(n,0);
 int l=0,r=0;
 for(int i=1; i<n; i++){
  if(i<=r) z[i]=min(z[i-l],r-i+1);
  while(i+z[i]<n and s[z[i]]==s[i+z[i]]) z[i]++;
  if(i+z[i]-1>r){
   l=i;
   r=i+z[i]-1;
  }
 }
 return z;
}
```